

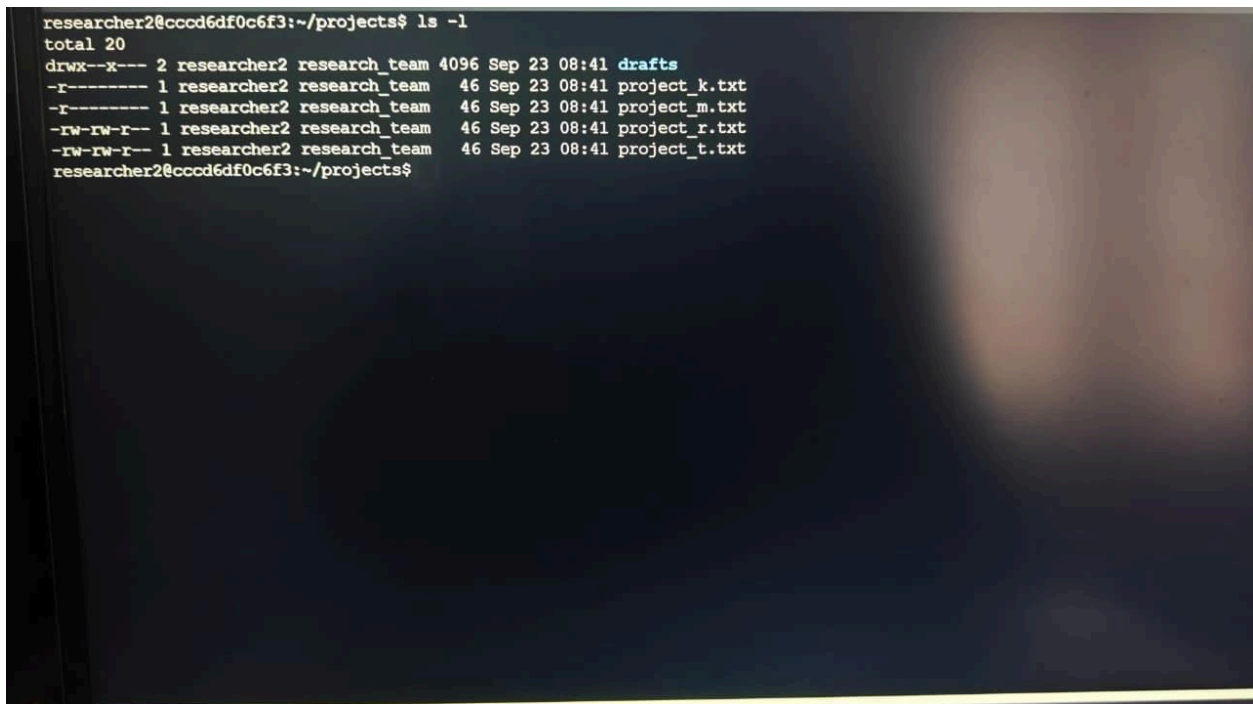
File permissions in Linux

Project description

In this project, I used Linux commands to manage file and directory permissions in a simulated research team environment. I checked existing permissions using `ls -l` and `ls -la`, then used `chmod` to restrict access so that only authorized users and groups could read, write, or execute specific files. This included securing a regular file, a hidden file, and a directory, ensuring the principle of least privilege was applied throughout

Check file and directory details

I navigated to the `~/projects` directory and used `ls -l` to view the permissions, ownership, and group assignment of each file. The output showed a 10-character permission string (e.g., `-rw-rw-r--`) along with the owner (`researcher2`), the group (`research_team`), file size, and last modified date for each file in the directory.



```
researcher2@cccd6df0c6f3:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team   46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team   46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$
```

Describe the permissions string

Each Linux permission string is 10 characters long. The first character indicates the file type (`-` for a regular file, `d` for a directory). The remaining 9 characters are split into three groups of three: owner, group, and others. Each group of three represents read (`r`), write (`w`), and execute

(x) permissions, in that order, with a - meaning that permission is not granted. For example, -rw-rw-r-- means the owner and group can read and write, while others can only read.

Change file permissions

I used the chmod command with symbolic notation to modify group permissions on project_m.txt, removing both read and write access (chmod g-r project_m.txt followed by chmod g-w project_m.txt), so that only the file owner retained access. I verified the change afterward using ls -l, confirming the permission string updated to -rw-----.

```
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 08:41 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 09:38 ..
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 .project_x.txt
drwx-x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$ chmod g-w .project_x.txt
researcher2@cccd6df0c6f3:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 08:41 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 09:38 ..
-rw----- 1 researcher2 research_team 46 Sep 23 08:41 .project_x.txt
drwx-x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$
```

Change file permissions on a hidden file

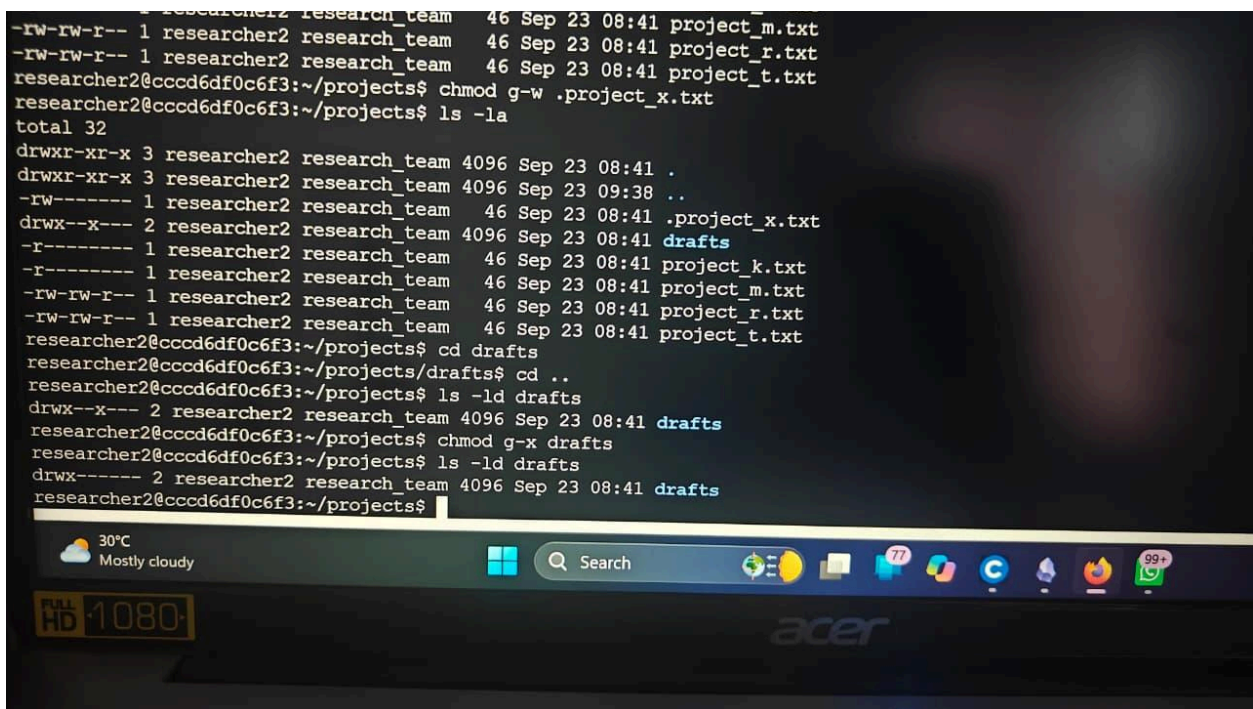
Hidden files (those starting with a .) don't show up with a regular ls -l, so I used ls -la to list them along with their permissions. I found that .project_x.txt had incorrect write permissions for both the owner and the group. Using chmod u-w,g-w,g+r .project_x.txt, I removed write access from both while preserving read access for the group, ensuring the archived file could be read but not modified.

```
researcher2@cccd6df0c6f3:~/projects$ ls -l
total 20
drwx--x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team  46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team  46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 08:41 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 09:38 ..
-rw--w--- 1 researcher2 research_team  46 Sep 23 08:41 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team  46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team  46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$
```

Change directory permissions

I checked the permissions of the drafts directory using `ls -ld drafts` (the `-d` flag shows the directory's own permissions rather than its contents). Since only the file owner needed execute access, I used `chmod g-x,o-x drafts` to remove execute permission for the group and others, restricting who could enter or list the contents of the directory.

```
researcher2@research_team 46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$ chmod g-w .project_x.txt
researcher2@cccd6df0c6f3:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 08:41 .
drwxr-xr-x 3 researcher2 research_team 4096 Sep 23 09:38 ..
-rw----- 1 researcher2 research_team 46 Sep 23 08:41 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_k.txt
-r----- 1 researcher2 research_team 46 Sep 23 08:41 project_m.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_r.txt
-rw-rw-r-- 1 researcher2 research_team 46 Sep 23 08:41 project_t.txt
researcher2@cccd6df0c6f3:~/projects$ cd drafts
researcher2@cccd6df0c6f3:~/projects/drafts$ cd ..
researcher2@cccd6df0c6f3:~/projects$ ls -ld drafts
drwx--x--- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
researcher2@cccd6df0c6f3:~/projects$ chmod g-x drafts
researcher2@cccd6df0c6f3:~/projects$ ls -ld drafts
drwx----- 2 researcher2 research_team 4096 Sep 23 08:41 drafts
researcher2@cccd6df0c6f3:~/projects$
```

The image shows a laptop screen with a terminal window open. The terminal displays a series of Linux commands and their outputs, including file permissions, directory changes, and file listings. The background of the terminal is dark, and the text is white. At the bottom of the screen, a Windows taskbar is visible, showing the date and time as 10:80, the weather as 30°C Mostly cloudy, and several application icons. The Acer logo is visible on the laptop bezel.

Summary

This project reinforced how Linux's permission system enforces access control at a granular level using the owner/group/other model. I practiced reading permission strings, applying the principle of least privilege with `chmod`, and verifying changes using `ls -l`, `ls -la`, and `ls -ld`. These are foundational skills for a SOC analyst role, since restricting unnecessary access is a core part of reducing attack surface and preventing unauthorized data exposure.